P1.1 — direct-Ir informed Berge search + P-graph extension audit

Aiko Seto

Created-at: 2026-09-01 20:25 CEST

Governance

`GOVERNING-STRATEGY: docs/hymeko-platform-strategy.md` · `STRATEGY-ID: HYMEKO-PLATFORM-2026-09-01-v` · `ROADMAP-TASK: P1.1` · `APPROVED-GRAPH-EDIT: direct-ir-informed-berge-search`. Strategy ID not bumped. **plan.pdf waiver**: no TeX engine on this host (tools.yaml `on_absent_tool: document_waiver_in_report`); `plan.tex/.tikz/.mmd` authored compilable, pdf not built.

ETA

≈4–5 h: promotion gate 15 min (done); reading + audit 45 min (done); plan bundle 30 min; baselines 30 min; A* + heuristics + policy implementation 90 min; oracles/fault-injection 60 min; benchmark 40 min; D*/LPA* decision + prototype 30 min; pgraph audit doc 20 min; V&V + reports + push 45 min. Basis: the search reuses the existing incidence primitives; the risk is entirely correctness (oracle parity) and the warm-alloc-free contract, both machine-gated.

Scope / goal

An *informed* Berge-search layer over the canonical `Ir` + exact sparse `IncidenceIndex`: Dijkstra/uniform-cost baseline, A* shortest Berge path, metric and hypergraph-discrete heuristics, a deterministic caller-owned workspace, warm-query allocation-free, and **no** materialised Levi/star/clique or second graph model. Plus a **read-only** architectural + performance audit of making `hymeko_pgraph` a true domain *extension* of `hymeko_graph` (general navigation/search owned by graph; classification/axioms/MSG/SSG/ABB/cost owned by pgraph). The D* Lite/LPA* line is **designed and benchmark-prototyped only**, not wired to production IR mutation (no canonical transaction/snapshot layer yet).

P1 promotion gate (§0, executed)

Verified 4 refs by `ls-remote`; ancestry `2ebb833`≺`5fe2c88`, `cbaadd4`≺`c35b9f2`; re-ran the P1 targeted gates (fmt/clippy/KAT/pgip) green; fast-forward-only promotion `core main` → `5fe2c88`, `graph main` → `c35b9f2`. New branch `feat/direct-ir-informed-berge-search` off the promoted graph main. Core production source untouched.

Semantic contract (see semantic-contract.md)

State space: implicit incidence `node` → `(arc,slot)` → `node`; forbidden: explicit Levi/star/ clique, dropping sign/weight/slot/duplicate, dense-id reinterpretation. Cost: typed, finite, non-negative (`NaN`/$\pm\infty$/negative = typed error), exit-independent-separable primary model ($c = w_{arc} + w_{entry}$) enabling arc-activate-once (O(incidences)); metric/general path relaxes per member. Policy: undirected / sign-compatible / directed producer→consumer / consumer→producer / domain — filter only, chosen once (no runtime mode boolean, no `Box<dyn>` in the hot loop). Order: `f64::total_cmp` then `(discrete_tiebreak, DeclId, HyperArcId, slot)`. Reopen: consistent- heuristic fast path; an inconsistent heuristic in optimal mode is a typed rejection (fail-closed), not a silent sub-optimal path.

Affected files (new; additive)

`hymeko_graph/src/direct_ir/informed.rs`	NEW: workspace, cost/heuristic/policy traits,
`hymeko_graph/src/direct_ir/mod.rs`	+`pub mod informed;` + re-exports
`hymeko_graph/tests/informed_search_vv.rs`	NEW: oracles, equivalences, boundary, fault-i
`hymeko_graph/tests/informed_search_bench.rs`	NEW: latency / expanded-state / heuristic-qu
`hymeko_graph/tests/informed_search_alloc.rs`	NEW: warm-query zero-alloc (isolated single-t
`docs/pgraph-as-graph-extension.md`	NEW: read-only audit narrative
`reports/2026-09-01-direct-ir-informed-berge-search.md`	NEW: P1.1 verification report

CORE.YAML items touched: **none** (`hymeko_graph` is core-listed in graph CORE.YAML; this is an **additive** new module under the `direct-ir-informed-berge-search` token — no existing public API changed, no dependency added). `hymeko_pgraph` not modified (read-only audit). Core repo untouched.

Test / coverage / benchmark matrix

Oracles: unit-cost = BFS `berge_shortest_path`; weighted = a test-only Dijkstra on the same implicit state space; tiny graph = exhaustive simple-Berge-path enumeration; metric = closed-form. Equivalences: `astar(Zero)`=Dijkstra, unit=BFS length, admissible=same cost. Boundary: 0/1/self/singleton/empty/disconnected, signs, f32/f64 edges, `NaN`/∞/negative rejection, arity 0/1/63/64/65 + large, declaration-order permutation, disjoint-union metamorphic. Fault-injection (no mutant committed, git-clean restore): heuristic overestimate, wrong predecessor slot, skipped reopen, broken tie-break, sign-policy inversion, duplicate-incidence dedup, stale workspace generation, dropped queue key-update — each caught by a named test. Benchmark: workload-matrix.md; per-cell expanded-state, queue ops, allocations, retained bytes, peak RSS, heuristic precompute (separate), time-to-first, total, path cost/hash. Coverage: new production lines by the V&V tests; touched-line target 100% or a line-justified fail-closed exception; before/after test population reconciled by name.

Performance budget / gates (§12)

Correctness first (optimal cost identical on every oracle cell). A* production-retention requires: warm query allocation-free; no materialised adjacency; on a metric-friendly or discrete-friendly

workload $\geq 2\times$ fewer expanded states *or* $\geq 20\%$ wall improvement vs Dijkstra; on an uninformative heuristic $\leq 10\%$ overhead vs the Dijkstra engine (or a separate zero-heuristic fast path); memory $O(\text{nodes} + \text{arcs})$; **no** $> 3\%$ regression in any untouched direct_ir op. The goal is that the *benchmark selects* where a heuristic pays — not that every heuristic always wins.

D* Lite / LPA* decision (§8)

Decision matrix: static+single-query → A*; same goals, changing cost/topology → LPA* candidate; moving start → D* Lite candidate; AND/OR producibility → separate domain model (not simple Berge D*). No production incremental state (no snapshot/transaction layer): a **benchmark-only** prototype may use two immutable before/after `Ir` snapshots + an explicit `ChangedIncidence` list (not called a canonical delta), query-workspace state only (`g, rhs, queue, version`), no second adjacency. Measure fresh-A* vs replan for 1/10/100/1000 updates + start movement; repaired = fresh Dijkstra/A*; report break-even query count. Honest **DEFERRED** if no clear win.

Rollback / HARD STOP

Additive on one feature branch; not promoted. Drop on: oracle disagreement, warm-query allocation, a forbidden artefact (Levi/star/clique on the standard path, second engine, `Box<dyn>` hot dispatch, `partial_cmp` on cost, an `IRDelta`/snapshot framework, production D*), a $> 3\%$ untouched-op regression, or any pgraph/core/Nagare/Hotaru production modification. **HARD STOP** at P1.1 end: no production PGRAPH migration, no P2 operator-precompute.

Indexed frontier continuation, 2026-10-12 01:49 +02:00

User approved all candidates in the Core algorithm/data-structure catalogue. Core first: shared Index/imports, incremental symbol counts and linear parent classification pass 193 library and 181 IR tests, plus Clippy. This continuation supersedes the original implementation stop, not the Git publication gate. The checkout advanced externally to f7ed8c2 (orthtree merge plus the earlier arena-test adaptation); freshly fetched feature upstream matches. Reuse this checkout and preserve unrelated work. No modifications in the target informed search file appeared during that integration.

Owner: existing `hymeko_graph/src/direct_ir/informed.rs`. Discovery found lazy BinaryHeap queues in top-k modules but no indexed/decrease-key heap; those path-state algorithms are not substitutes. The current search already keys g, predecessor and settled state by DeclId, making one frontier entry per ID valid. Retain Entry/Key ordering and the one production A-star/Dijkstra entry point.

Use a Vec of Entry plus a full-width inverse position Vec. Membership checks the position against live heap length and the entry's DeclId. Clear only resets the trivial-entry Vec length; stale inverse entries are harmless. Ensure both arrays for the declaration domain before search. Insert/update/pop are O(log N), clear/membership O(1), live frontier at most N. Preparation and memory are O(N), even when the actual frontier is small: this is an explicit tradeoff. No further heap growth inside search. Existing generation-stamped state still has a wrap clear; this change does not remove that separate cost.

Push statistics continue counting successful relaxations plus the start entry, including priority updates. Pops/stale-pops count actual extractions and decrease. Preserve path costs, policies, tie

choices and errors. No signature or dependency change. Touched: this bundle, CORE.YAML, SYMBOLS.md if generated output changes, and `hymeko_graph/src/direct_ir/informed.rs`. Capture the source before edits. Private queue tests use the minimum scan over one optional entry per ID as a small independent oracle, not another search engine. Cover both key directions, root/last removal, clear, regrowth and reuse. Run existing public path/oracle and allocation checks, Graph correctness/strict Clippy, then Web/WASM consumers. Use one local Core identity; temporary lockfiles restored only after hash checks. No timing matrix. Resource cap 16 GiB. ETA 15 minutes (implementation 5, tests 5, shared PDF/report 5). Roll back only these named hunks. TeX is available here; build plan.pdf before edits. The original PDF-tool waiver is historical.

Verification closure, 2026-10-12 02:05 +02:00

Implemented one private indexed frontier. Three new tests cover 6,144 mixed queue operations against a minimum-scan oracle, equal-key payload replacement, and an integrated improved-route query with four updates, three pops and no stale pop. Full workspace lib/tests: 644 passed, 30 ignored. Strict Clippy, formatting and symbol index checks pass. Web/WASM's 42 native tests and wasm32 check pass with one local Core/Graph identity; original lockfiles restored.

Duplication gate remains a pre-existing failure: the checked-out HEAD f7ed8c2 and candidate both measure 577 clones / 13,157 lines, against stored 547 / 12,629. The existing detector was used with the HEAD version of the single changed source substituted only in memory. Zero added duplication; baseline unchanged. Evidence and shared PDF: `reports/2026-10-12-indexed-frontier/` and `reports/2026-10-12-indexed-frontier.pdf`. No latency gain is claimed. The original source SHA-256 starts `0e8d885d2e64847f`. No public API or dependency change here; observable pops now reflect fewer stale extractions. No commit or publication by this task; preserve the externally integrated branch.